BANGALORE UNIVERSITY

Jnana Bharathi Campus, Bangalore-560056

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

A Project Report On

"PLACEMENT PRO"

Submitted by

HEMANTH R (P03NK23S126036)

VINAYAKA K S (P03NK23S126048)

Under the guidance of

DR. M HANUMANTHAPPA

Senior Professor and Coordinator

Department of Computer Science and Applications

Towards

MINI PROJECT

Prescribed by the BANGALORE UNIVERSITY

MASTER OF COMPUTER APPLICATIONS

For the academic year **2024-2025**

BANGALORE UNIVERSITY

Jnana Bharathi Campus, Bangalore-560056

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

A Project Report On

"PLACEMENT PRO"

CERTIFICATE

This is to certify that **HEMANTH R (P03NK23S126036) and VINAYAKA K S (P03NK23S126048)** has satisfactorily completed the project titled **"PLACMENT PRO"** towards Mini project Lab prescribed by the Bangalore University for the 3rd Semester Master of Computer Application course in academic year 2024-2025.

DR. M HANUMANTHAPPA

Guide, Senior Professor and coordinator

Examiners:

1)

2)

DECLARATION

I hereby declare that the project report, titled **"PLACMENT PRO"**, is prepared in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications of Bangalore University for the academic year 2024-2025. Further, the matter embodied in dissertation has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

HEMANTH R (P03NK23S126035)

VINAYAKA K S (P03NK23S126048)

DATE:18/06/2025

PLACE: Bangalore

ACKNOWLEDGEMENT

The satisfaction that I feel at the successful completion of the project titled **"PLACEMENT PRO"** would be incomplete if I do not mention the names of the people whose valuable guidance and management has made this project work a success.

It is a great pleasure to express my gratitude and respect to all those who inspired and helped me in completing this project.

It is a pleasure to thank my internal guide, **DR. M HANUMANTHAPPA**, Senior Professor and Coordinator of the department of Computer Science & Application, Bangalore University, for his valuable assistance and cooperation, who has given me ideas and guidance to make the report a highly useful & knowledge-based experience.

It is also a pleasure to express my gratitude to all Teaching and Non-teaching staff members of the Department of Computer Science and Application for their encouragement and providing valuable requirements.

With a deep sense of indebtedness, I convey my heartiest thanks to my parents who have taken effort and given me such an opportunity. Their love and blessing has given me strength to acquire knowledge and gain experience in my life. As well as my friends who have helped for this accomplished task.

ABSTARCT

The "PLACMENT PRO" is a comprehensive web-based application designed to streamline and automate the entire campus recruitment process. Built using the powerful Python-based Django web framework, this system serves as a centralized platform connecting three key stakeholders: the college's Training and Placement Officer (Admin), students, and company representatives.

The platform emphasizes security, scalability, and ease of use, enabling educational institutions to save significant time, reduce manual errors, and improve communication during the placement season. Core functionalities include a role-based authentication system, dedicated dashboards for each user type, student profile management with resume uploads, company job postings with eligibility filters, and an automated application tracking system. The system enhances the efficiency of the placement process through automation, centralized data handling, and insightful analytics.

CONTENTS

1. INTRODUCTION

PLACEMENT PRO Using Django

In today's competitive academic and professional landscape, educational institutions are increasingly adopting digital tools to enhance efficiency and bridge the gap between students and industries. A critical aspect of this is the campus placement process, which traditionally involves significant manual effort. The "PLACEMENT PRO," built with the Django web framework, automates and simplifies this complex task.

This system is a web-based application designed to streamline the entire recruitment lifecycle within an educational institution. Manually managing student data in spreadsheets, coordinating with numerous companies via email, and tracking application statuses is a time-consuming and error-prone process. This project offers a centralized platform where student profiles, company information, and job postings are stored and managed efficiently.

By automating the placement process, the system significantly reduces administrative overhead and minimizes errors. It provides role-based access for the Training and Placement Officer (Admin), Students, and Company HR, ensuring that each user only interacts with the features relevant to them. With its clean, responsive interface and scalable architecture, the tool is well-suited for the dynamic needs of a modern college placement cell.

1.1 Overview

This web-based system allows the Training and Placement Officer (TPO) to manage the entire placement process from a central dashboard. It reduces manual effort, ensures transparent communication, and provides data-driven insights. A centralized database for students and companies helps maintain consistency and simplifies coordination between all parties.

1.2 Why It’s Needed

Manually creating and managing placement drives is a time-consuming process that often results in several issues, such as students applying for jobs for which they are not eligible, communication gaps between the TPO and students, and a lack of data for year-on-year analysis. This not only affects the efficiency of the placement cell but also increases the workload for administrators and faculty.

1.3 Project Goals

The system offers secure access through a role-based login for Admins, Students, and Companies. It features well-organized modules where students can manage their profiles and apply for jobs, companies can post openings and shortlist candidates, and the admin can oversee

the entire process. The application supports automated eligibility checks, status tracking, and notifications to make the process faster and more efficient.

1.4 Benefits

The system saves significant time and effort by automating registrations, applications, and status updates. It helps reduce errors in eligibility checking and ensures that communication is clear and timely. By providing a transparent platform for all stakeholders, it supports a fair and efficient placement process. Additionally, the application is easy to use and scalable, making it suitable for institutions of various sizes.

2. OBJECTIVES

The "PLACMENT PRO" is designed with several key objectives to streamline and enhance the campus recruitment process:

- **Centralized Database:** To develop a comprehensive database that stores all student academic and personal data, company profiles, and job postings in one secure, centralized location.
- **Role-Based Access Control:** To implement a secure login system using Django's authentication that provides distinct roles and permissions for Admin (TPO), Student, and Company users.
- **Automated Job Application Workflow:** To create a seamless workflow where companies can post jobs with specific eligibility criteria, and students can view and apply only for the jobs they are eligible for.
- **Real-Time Status Tracking:** To enable students and companies to track the status of job applications in real-time (e.g., Applied, Shortlisted, Interview Scheduled, Offered).
- **Admin Oversight and Management:** To provide the TPO with a powerful admin dashboard to manage all users, approve job postings, oversee interview schedules, and generate analytical reports.
- **Scalability and Maintainability:** To design a system with a clean, modular architecture that can handle a large volume of data and can be easily updated with new features in the future, such as automated email notifications and advanced analytics.

3. PROJECT DESCRIPTION

This is a web-based application built with Django to help college placement cells manage and automate their entire recruitment process. It uses a centralized database and provides separate, feature-rich panels for Admins, Students, and Companies.

3.1 Core Features

The system allows three types of users (Admin, Student, Company) to securely log in using Django's authentication system. Once logged in, each user is directed to a dashboard tailored to their role, with access to specific functionalities.

3.2 Student Module

Students can register and create a detailed profile, including academic scores, skills, and personal information. They can upload their resumes, browse a list of available jobs for which they are eligible, apply with a single click, and track the status of their applications throughout the hiring process.

3.3 Company Module

Company representatives can register their organization, create a company profile, and post new job openings with specific eligibility criteria (e.g., required branches, minimum CGPA). They can view a list of all students who have applied for their jobs, download their resumes, and manage the shortlisting process.

3.4 Admin (TPO) Module

The Admin has complete oversight of the system. From their dashboard, they can manage all student and company accounts, approve or reject new job postings from companies, view system-wide analytics, and generate reports on placement statistics.

3.5 Workflow

A company posts a job, which is then sent to the Admin for approval. Once approved, the job becomes visible to all eligible students. Students can then apply for the job. The company can view all applicants for their job, shortlist candidates, and schedule interviews. The student is notified of status changes and interview schedules.

3.6 Example Use Case

A student from the "Computer Science" branch with an 8.5 CGPA logs in. They see a "Software Engineer" job posting from TechCorp that requires a CGPA of 8.0 or above and is open to "CSE" and "ECE" branches. Since they are eligible, they click "Apply." Later, the TechCorp HR logs in, sees the student's application, reviews their profile and resume, and shortlists them for a

technical interview. The student receives a notification that their application status has changed to "Shortlisted."

4. SYSTEM ANALYSIS

This is a web application built with Django to automate the campus placement process. It offers a centralized platform for students, companies, and the placement officer. It replaces manual, error-prone processes with an efficient, role-based system.

4.1 Problems in Existing System

The traditional manual process of managing placements involves scattered data on spreadsheets, cumbersome email communication, and a lack of a centralized system for tracking applications. This can lead to students missing opportunities, administrative errors in eligibility checking, and difficulty in generating accurate placement reports.

4.2 Proposed Solution

The proposed system features secure login for three distinct user roles to ensure authorized access. It includes a comprehensive database for all student and company data. It supports automated job listings with eligibility filters, real-time application tracking, and an admin dashboard for efficient management and analytics

4.3 System Requirement

4.4 Functional

- **User Authentication:** Secure registration and login for Admin, Student, and Company roles.
- **Profile Management:** Students and companies can create and update their profiles.
- **Job Management:** Companies can post, manage, and view applicants for their jobs. Admins can approve or reject job postings.
- **Application System:** Students can view eligible jobs and apply. Application status must be trackable.
- **Admin Control:** Admins can manage all users and have access to system-wide data and reports.

4.5 Non-Functional

- **Security:** The system incorporates features like CSRF protection, secure session handling, and password hashing.
- **Usability:** It offers a clean, responsive, and user-friendly interface built with HTML, CSS, and Bootstrap.

- **Scalability:** The application is designed to accommodate a growing number of users, companies, and job postings.
- **Performance:** It is optimized for speed through efficient database queries using Django's ORM.

4.6 Feasibility Summary

- **Technical:** The system is built using open-source technologies like Django and Python, making it deployable on standard web servers without requiring specialized hardware.
- **Operational:** It is easy to use and helps reduce the workload of the placement cell by automating repetitive tasks.
- **Economic:** It is highly cost-effective as it does not involve any licensing fees for its core technologies, making it an affordable solution for educational institutions.

4.7 Benefits

The system offers fully automated workflows for job posting and applications. It ensures a fair and transparent process for all users. Role-based secure access controls protect the system's integrity. It also supports exporting data for reporting and record-keeping.

4.8 Limitations

The system requires a stable internet connection for all users. The initial setup requires a one-time effort to bulk upload existing student and company data. The user interface, while functional, can be further enhanced for a richer user experience.

5. SYSTEM DESIGN

5.1 MVT Architecture

The system is built on Django's Model-View-Template (MVT) architecture.

- **Model:** Defines the database structure, including tables for User, StudentProfile, CompanyProfile, JobPosting, and Application.
- **View:** Handles the application's business logic, processes user input from URLs, interacts with the models to fetch or save data, and renders the appropriate template.
- **Template:** Consists of HTML files embedded with Django Template Language (DTL) to display data dynamically to the users.

5.2 Core Modules

- **Authentication Module:** Handles user registration, login, logout, and role-based access control using Django's built-in auth system.
- **Student Module:** Contains all functionalities for students, including profile management, job listing, and application tracking.
- **Company Module:** Contains all functionalities for companies, including profile management, job posting, and applicant management.
- **Admin Module:** An admin panel that allows the TPO to manage the entire system, approve jobs, and view analytics.

5.3 Database Design (SQLite/PostgreSQL via Django ORM)

The system uses several key database tables managed via Django's ORM: User (stores login credentials and roles), StudentProfile (stores student-specific data), CompanyProfile (stores company-specific data), JobPosting (stores job details), Application (links students to jobs), and InterviewSchedule.

5.4 Data Flow Diagram (DFD)

- **Level 0 (Context):** A user (Admin, Student, or Company) interacts with the system to log in, manage their data, and perform role-specific actions (e.g., apply for a job, post a job).
- **Level 1 (Processes):** Login → Role-Based Dashboard → Perform Actions (e.g., View Jobs, Manage Applicants) → Database Interaction → Render Result.

5.5 User Interface Design

The system includes a public-facing homepage and login page. After login, each user is presented with a dashboard tailored to their role. Forms are used for all data entry (profiles, jobs), while tables and cards are used to display lists of data (job listings, applicants). The UI is built with HTML, CSS, and Bootstrap 5 to be fully responsive.

5.6 Scalability & Extensibility

The system features a modular design that allows for easy future upgrades, such as adding a real-time chat module or integrating advanced analytics. It is designed to support a large number of users and can be scaled by upgrading the server and database backend from SQLite to PostgreSQL.

6. PROJECT REQUIREMENTS

Components and Technology Used

The project is developed using Django, a high-level Python web framework that facilitates rapid development and clean, maintainable code. MySQL is used as the backend database to store and manage structured data efficiently. The frontend is built using HTML and CSS for layout and design, JavaScript for adding interactivity, and Django Templates for rendering dynamic content. User authentication and authorization are handled using Django's built-in authentication system, ensuring secure login and user session management. Additionally, the Django Admin Panel is utilized as the admin interface, providing a powerful and user-friendly way to manage application data and users without requiring additional coding.

7. FEATURES

User Login & Roles

- Secure login required for all users to access the system.
- Three distinct user roles: Admin (TPO), Student, and Company.

Admin Features

- Comprehensive dashboard with system-wide statistics.
- Full CRUD (Create, Read, Update, Delete) management of Student and Company accounts.
- Approval queue for moderating new job postings.
- Generation of data reports in CSV format.
- Bulk upload functionality for student and company data via CSV.

Student Features

- Secure registration and login.
- Personalized dashboard with application and interview stats.
- Create and manage a detailed profile with resume upload.
- View a filtered list of jobs for which they are eligible.
- Apply for jobs with a single click and track application status in real-time.

Company Features

- Secure registration and login for company representatives.
- Personalized dashboard with job and applicant stats.
- Create and manage a public-facing company profile with a logo.
- Post new job openings with specific eligibility criteria.
- View and manage a list of applicants for each job.
- Shortlist and manage candidates for interviews.

System Features

- Simple and intuitive interface built with HTML and Bootstrap 5.
- Use of Django's built-in admin panel for easy backend management.
- Role-based access control to ensure users only see relevant information.

Security

- Passwords are not stored in plain text and are managed securely using hashing.
- Protection against common web vulnerabilities like CSRF.
- Access to all panels is limited strictly to authenticated users.

8. IMPLEMENTATION

Project Setup

The system is built using Python and the Django framework. The project was initiated with the django-admin start project placement system command, and the main application was created with python manage.py start app core.

Database

The system uses SQLite as the default Django database for development. Models for User, Student Profile, Company Profile, Job Posting, etc., were defined in core/models.py. The database schema was created and updated using the python manage.py make migrations and python manage.py migrate commands.

User Authentication

The system utilizes Django's built-in login, logout, and password management functionality. Custom registration views were created to handle the sign-up process for students and companies, assigning specific roles to control permissions.

Module Implementation

- **Student Module:** View functions were created to handle profile updates, job listings with eligibility filtering, and application tracking.
- **Company Module:** View functions were implemented to allow companies to post jobs, view lists of applicants for each job, and change their application status.
- **Admin Module:** Views were built to provide the TPO with management control over users and job postings, as well as to generate reports and handle bulk data uploads.

Frontend & Templates

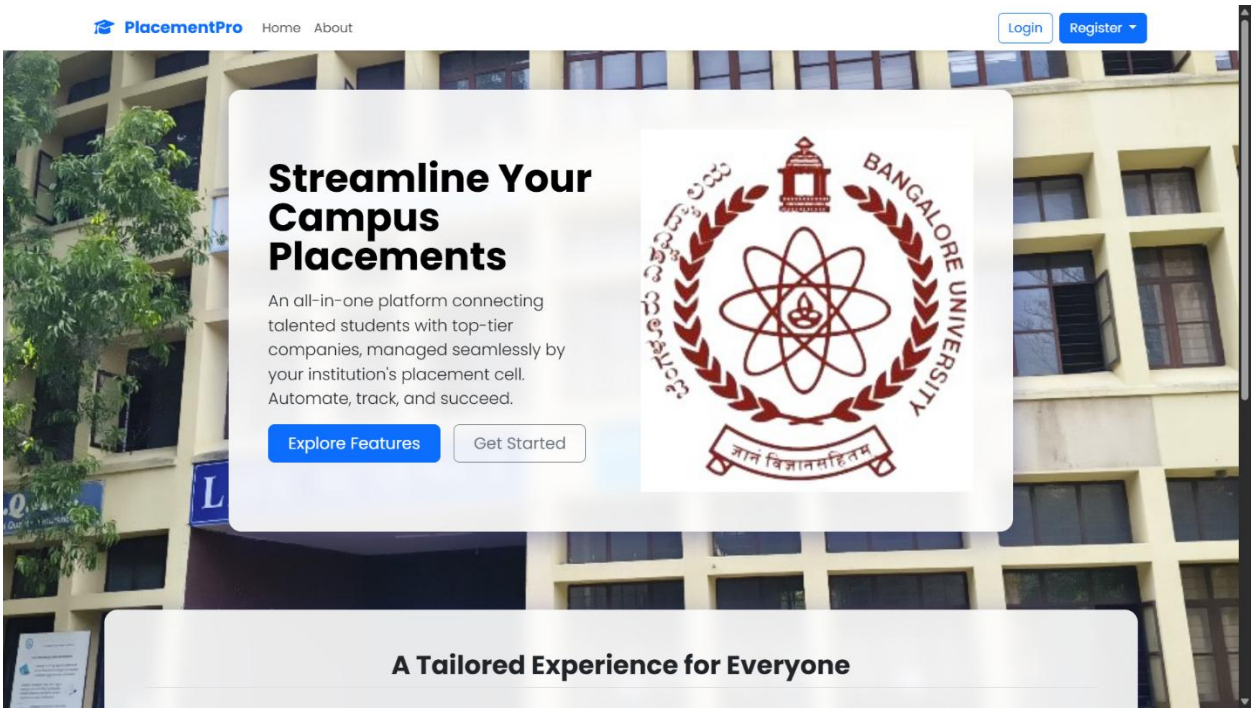
The system uses the Django Template Language combined with HTML, CSS, and Bootstrap 5 to render dynamic content and provide a responsive user interface. django-crispy-forms was used to render clean, styled forms.

Running the App

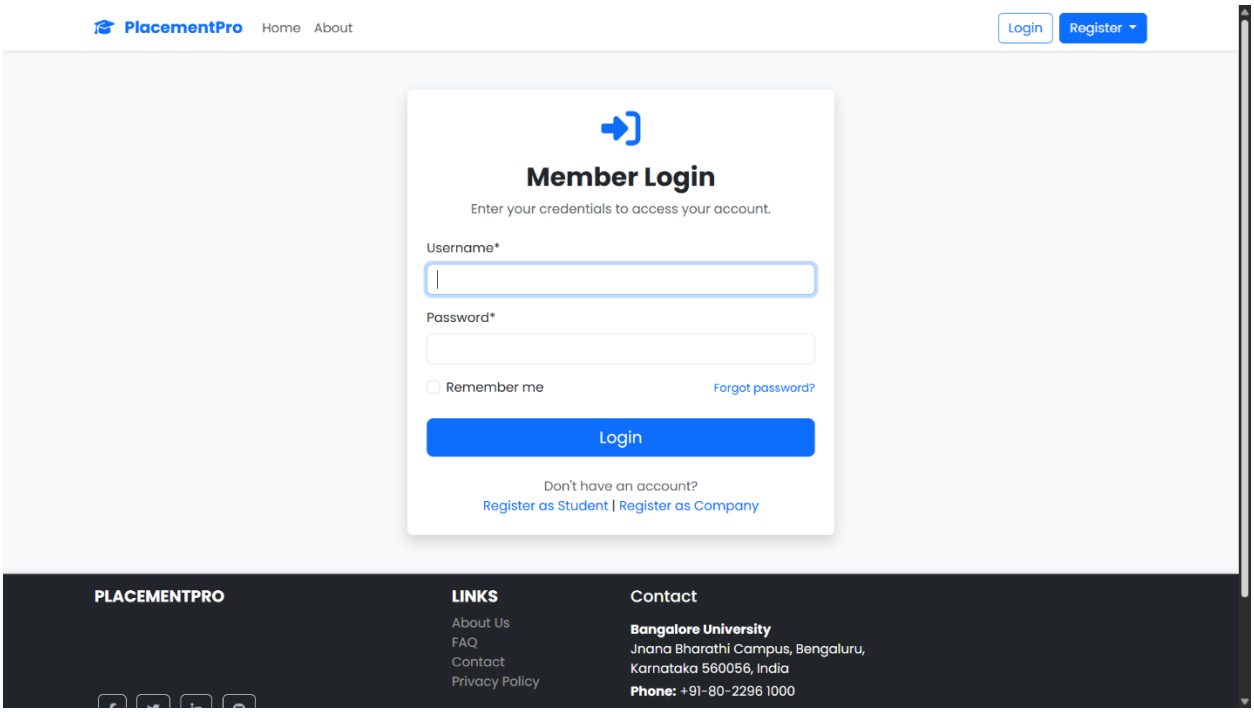
To start the development server, the command python manage.py runserver is used. The application can then be accessed locally at http://127.0.0.1:8000/.

9. SNAPSHOTS

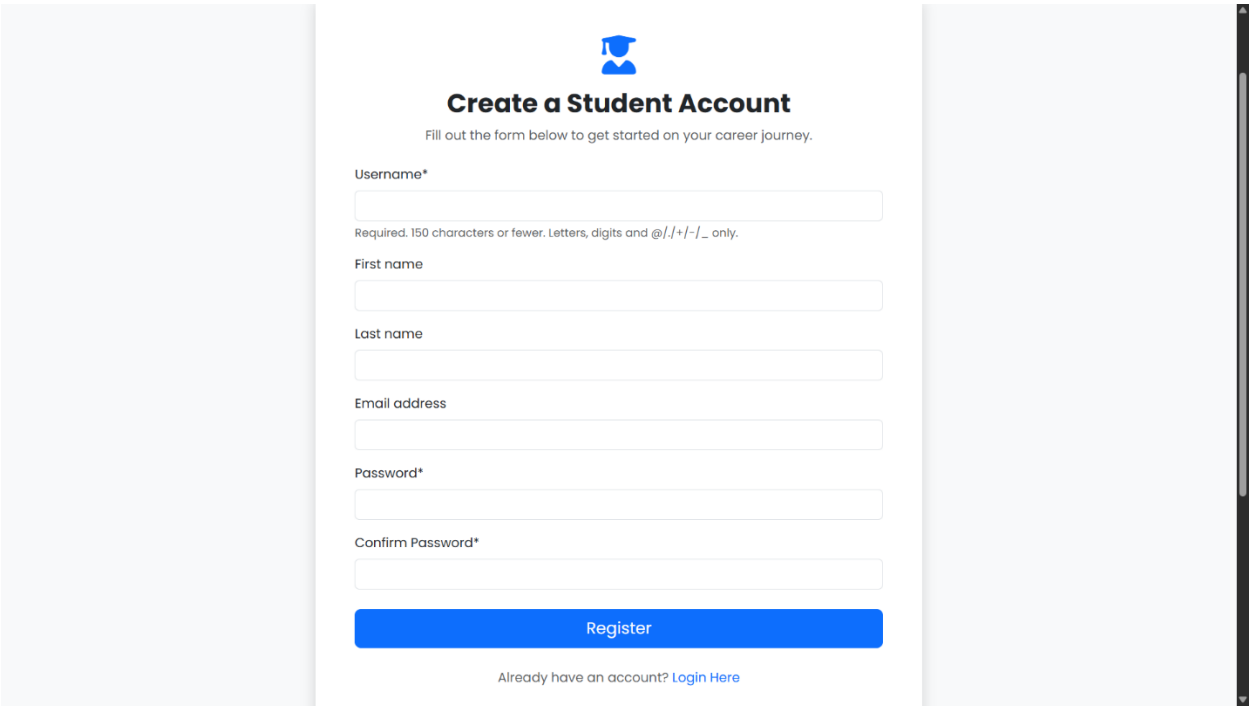
1. **Homepage:** The main landing page for all users.



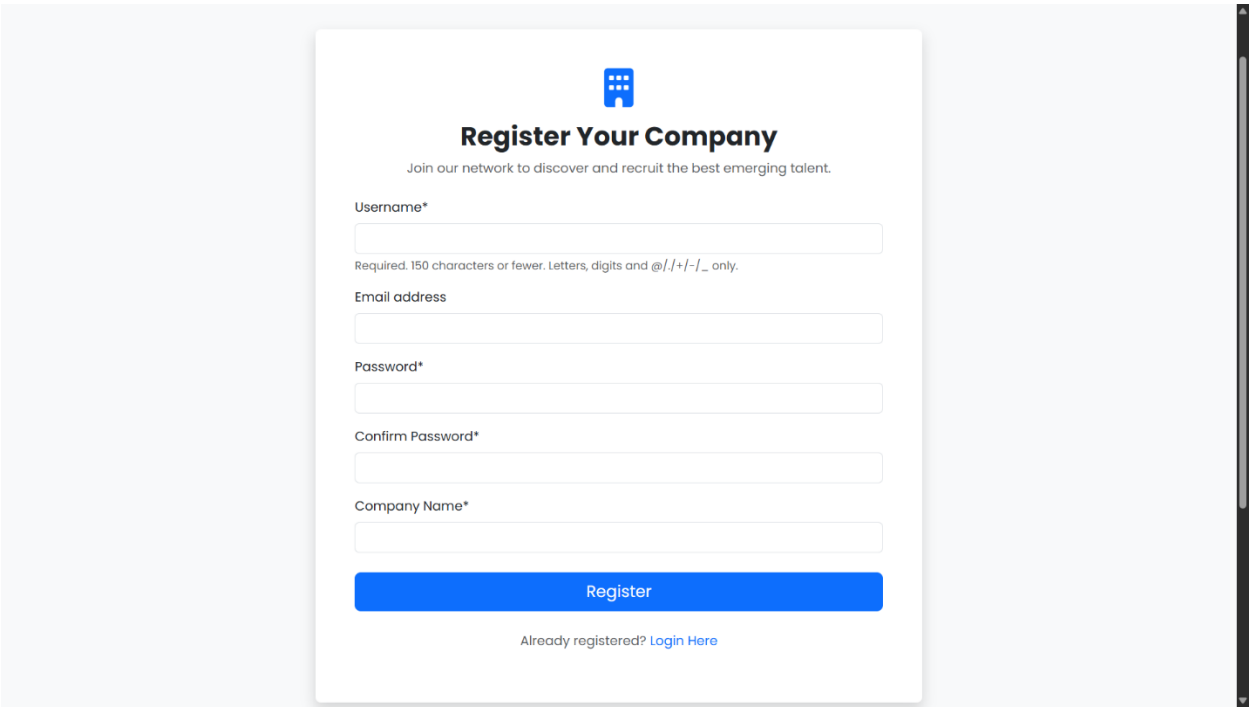
2. **Login Page:** The form for users to log in.



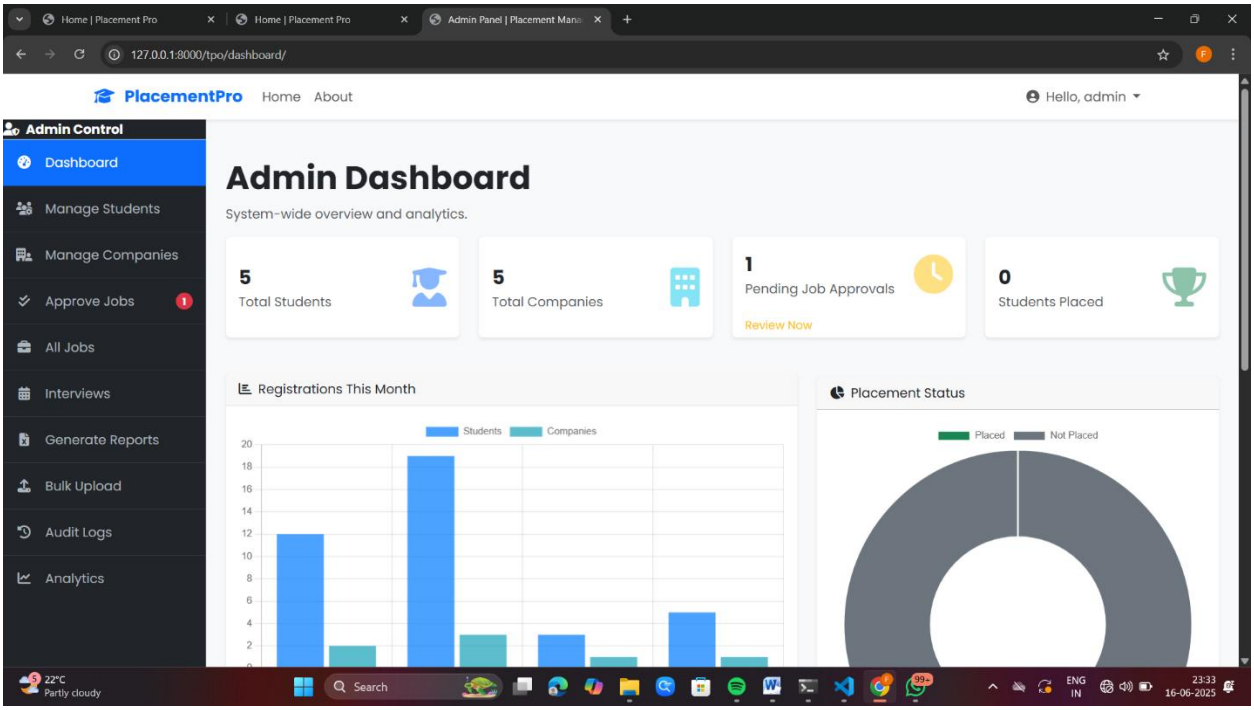
3. **Student Registration Page:** The sign-up form for students.



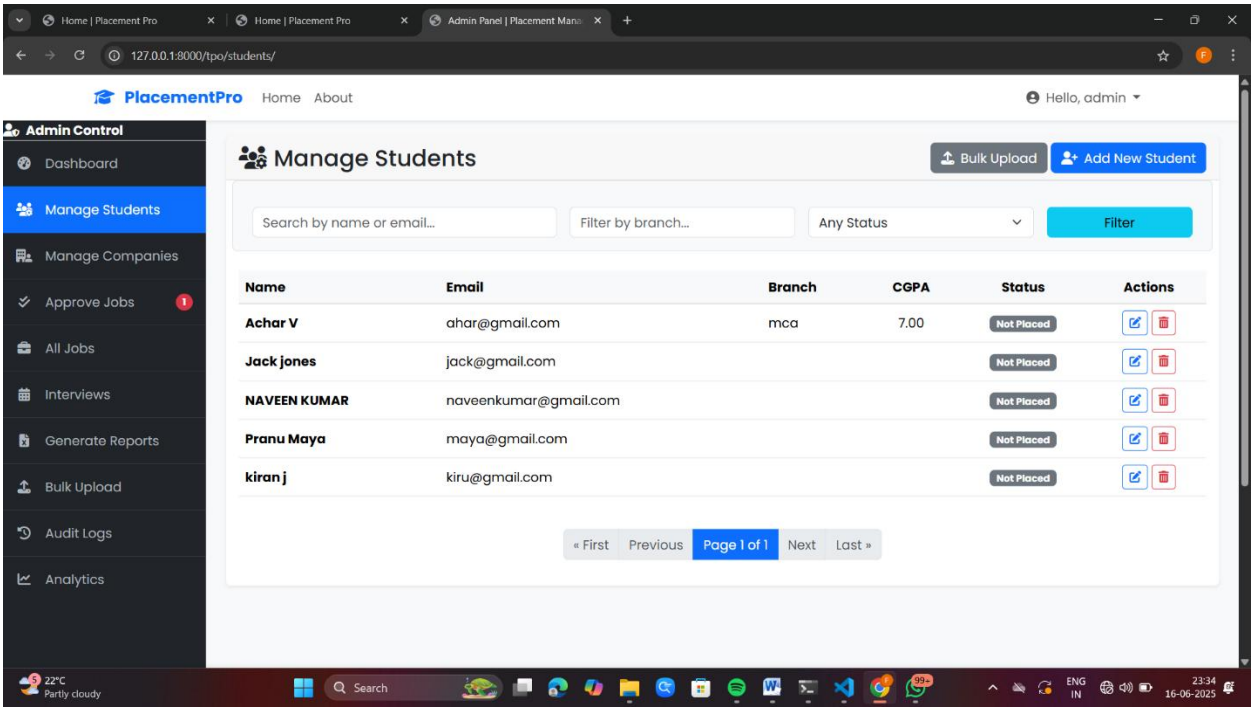
4. **Company Registration Page:** The sign-up form for companies.



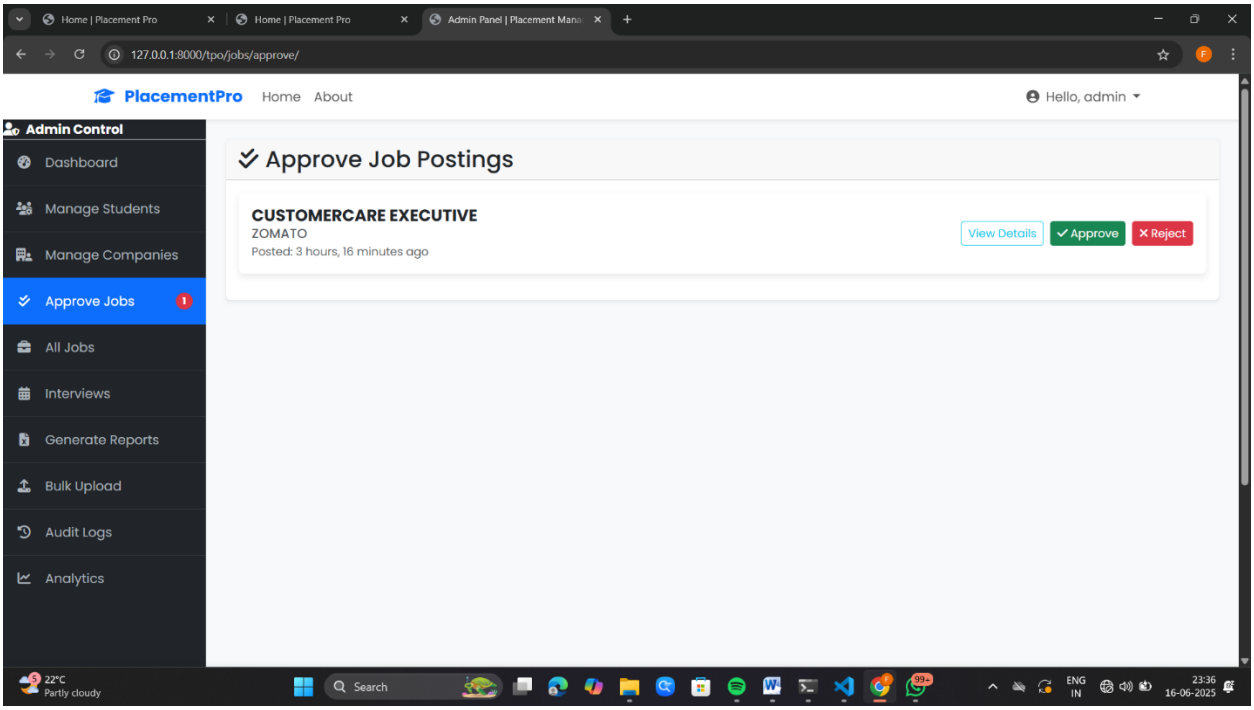
5. **Admin Dashboard:** The main dashboard for the TPO, showing statistics and charts.



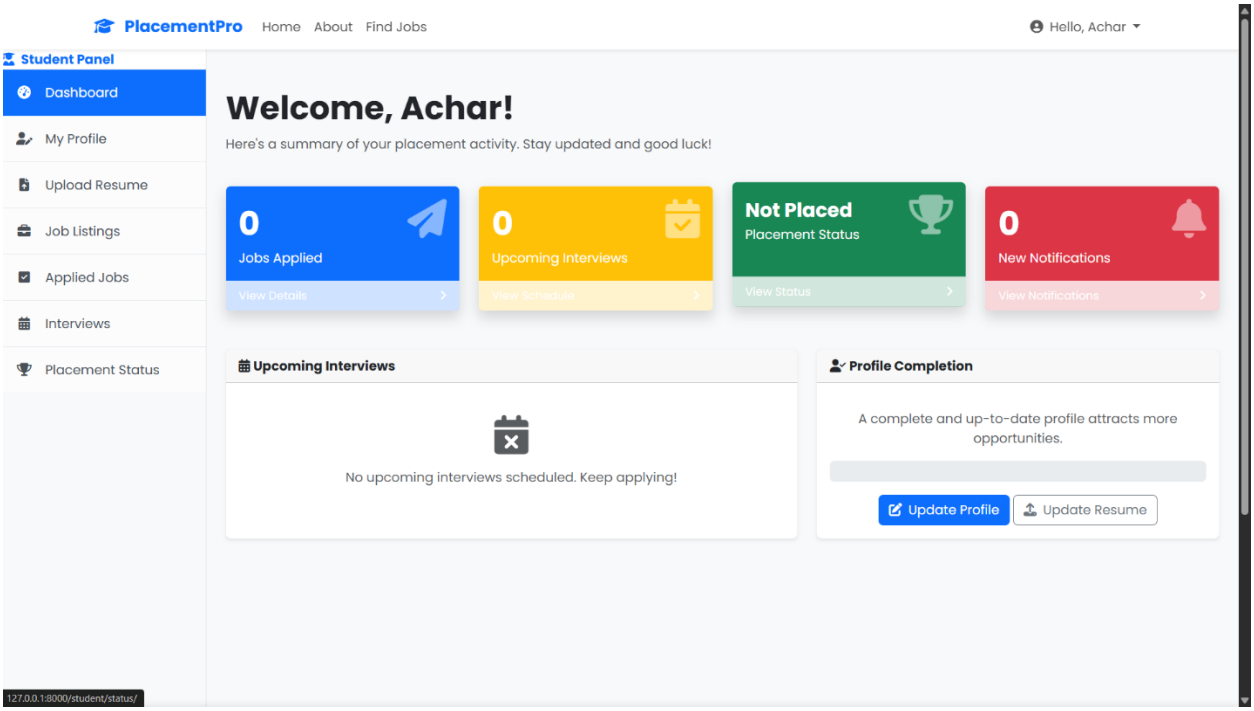
6. **Manage Students Page (Admin):** The table view showing the list of all students.



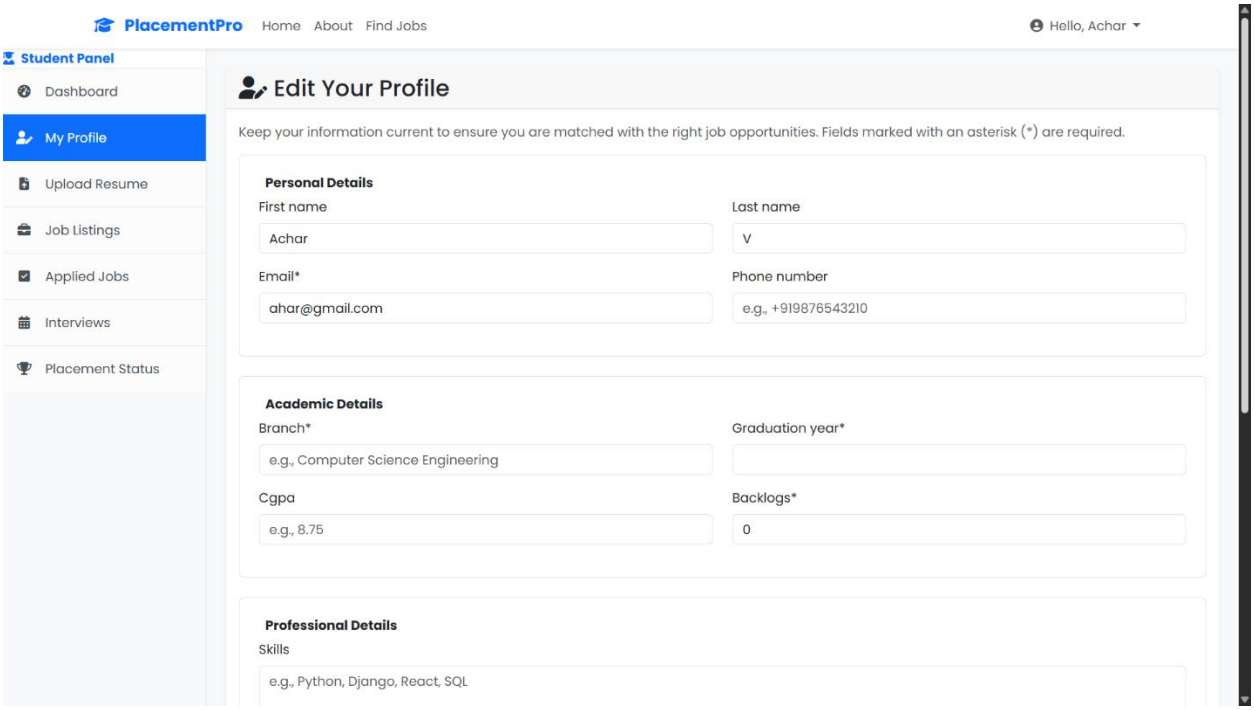
7. **Approve Jobs Page (Admin):** The queue of pending jobs waiting for approval.



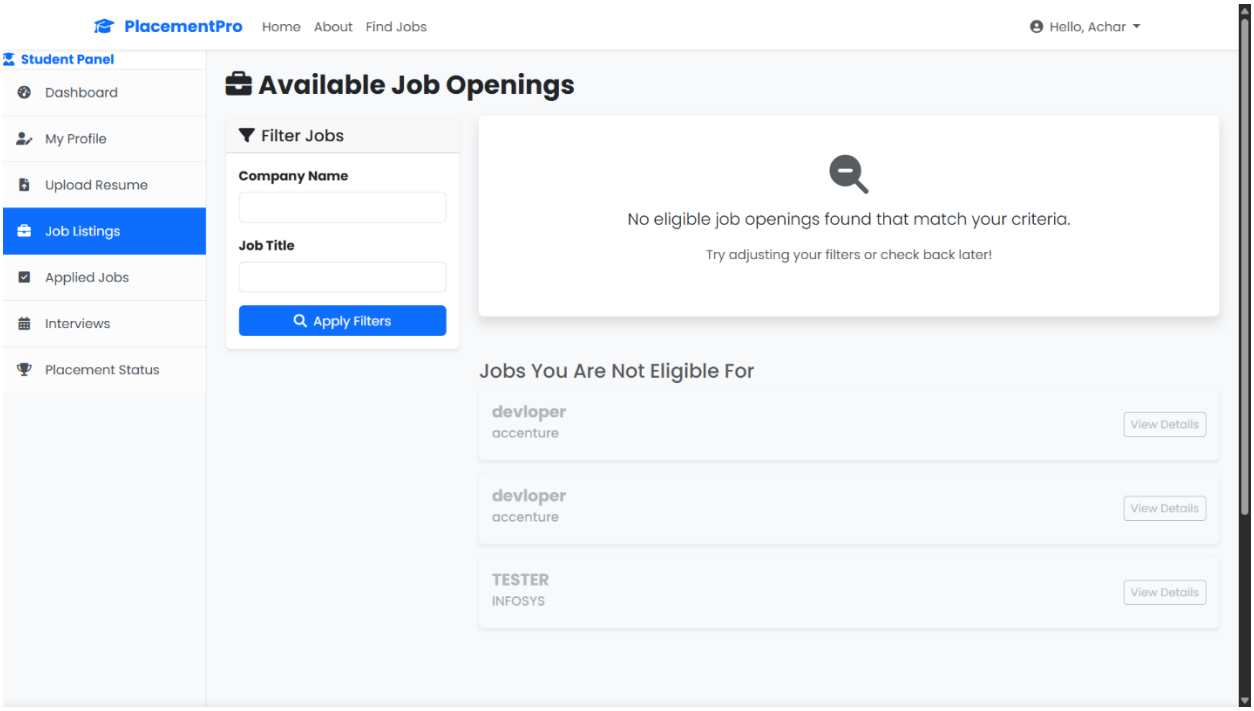
8. **Student Dashboard:** The personalized dashboard for a logged-in student.



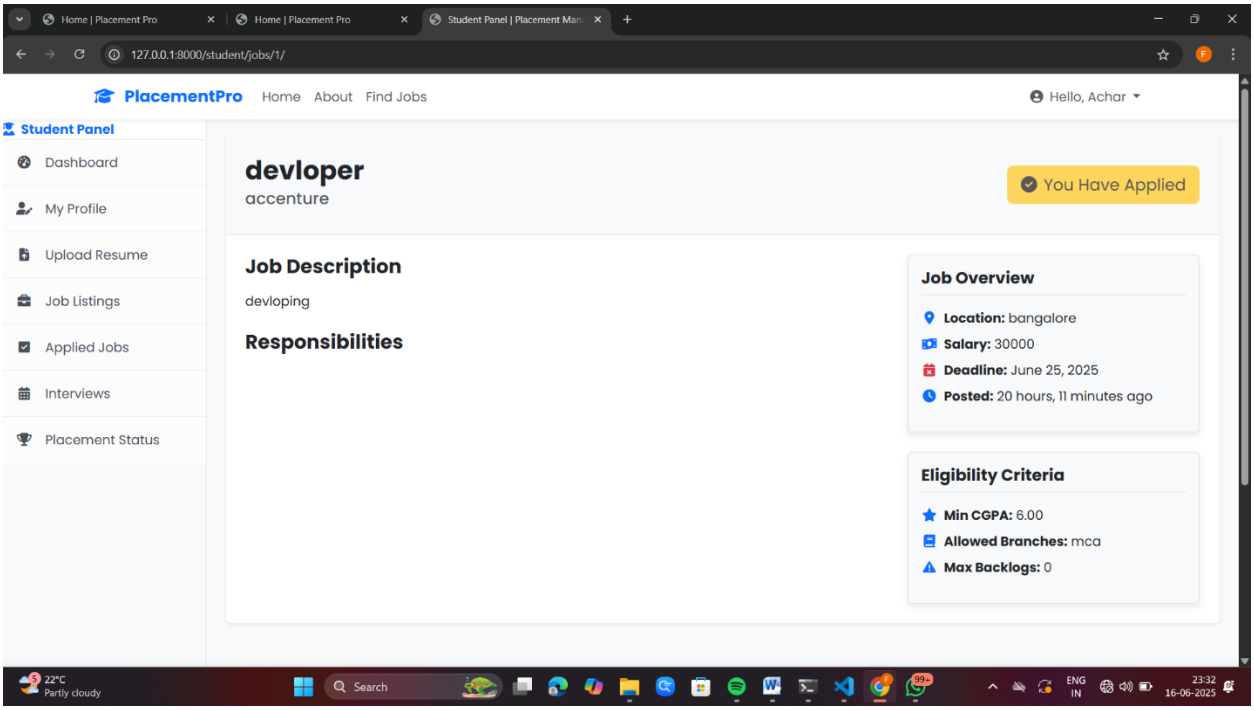
9. **Student Profile Page:** The form showing the student's editable profile.



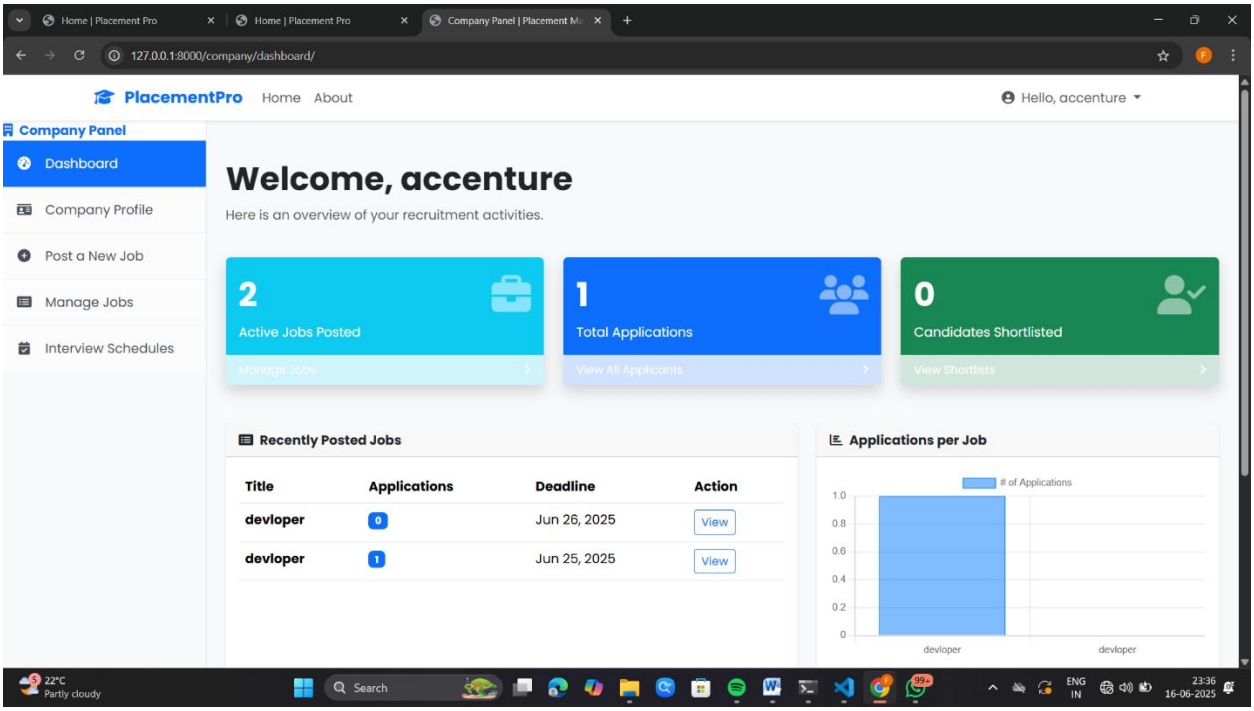
10. **Job Listings Page (Student):** The list of jobs available to a student.



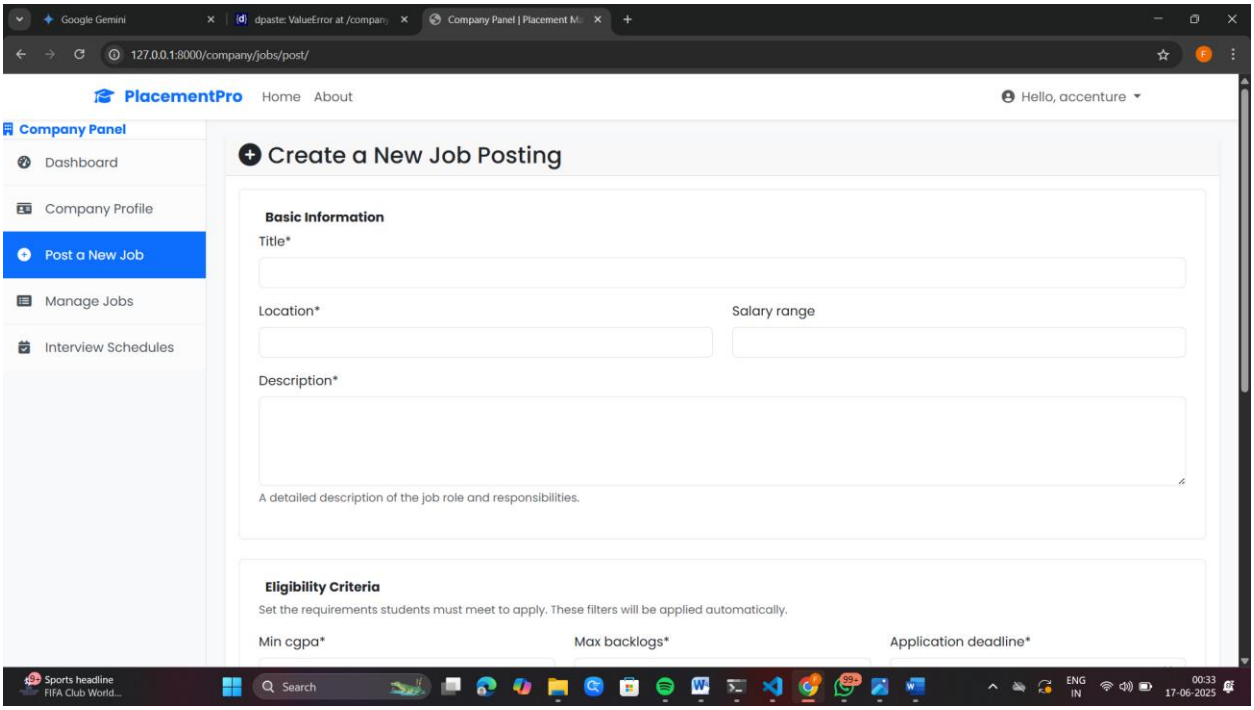
11. **Job Detail Page (Student):** The detailed view of a single job.



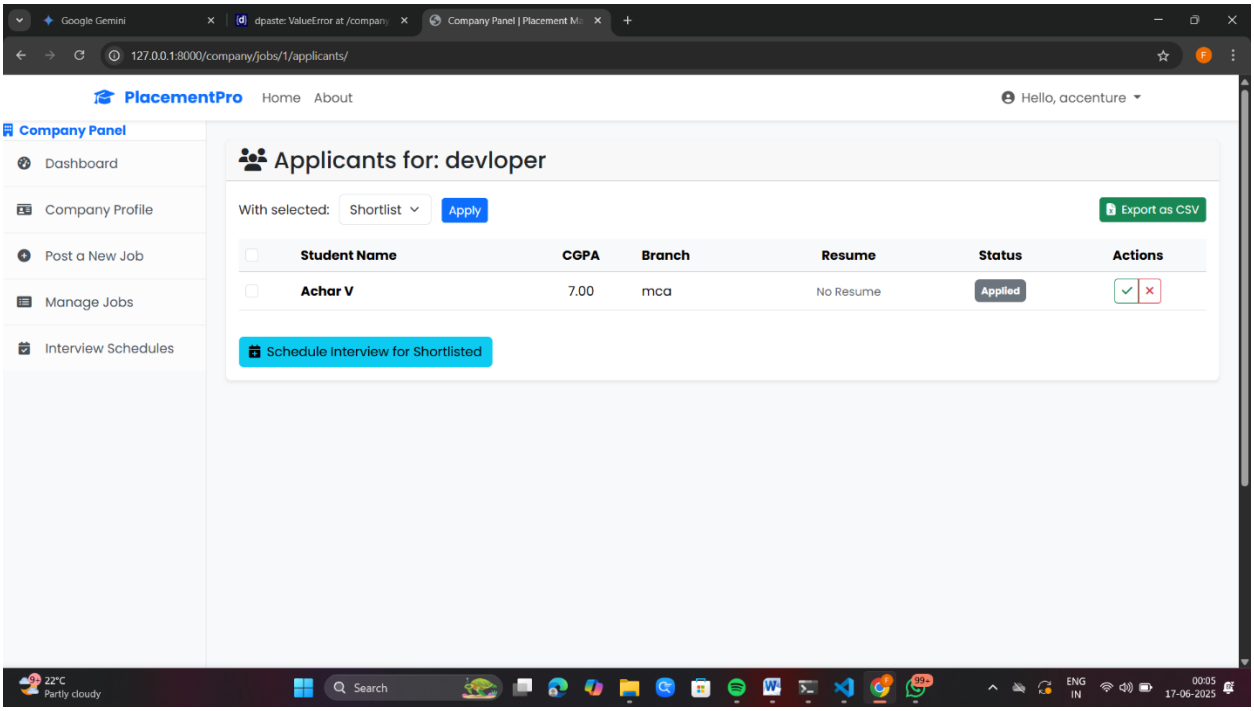
12. **Company Dashboard:** The personalized dashboard for a logged-in company user.



13. **Post a Job Page (Company):** The form for a company to create a new job posting.

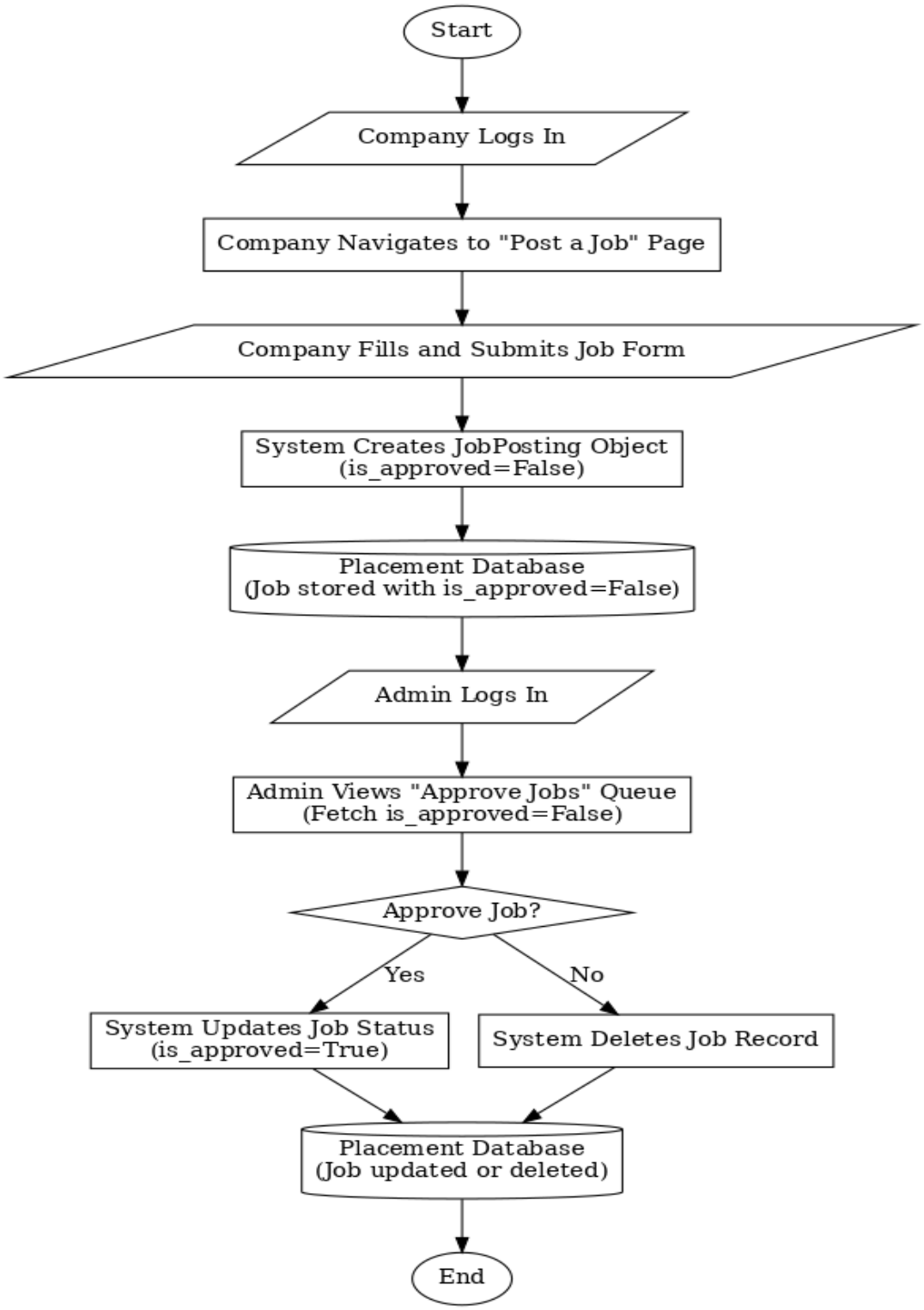


14. **Job Applicants Page (Company):** The list of students who have applied for a specific job.

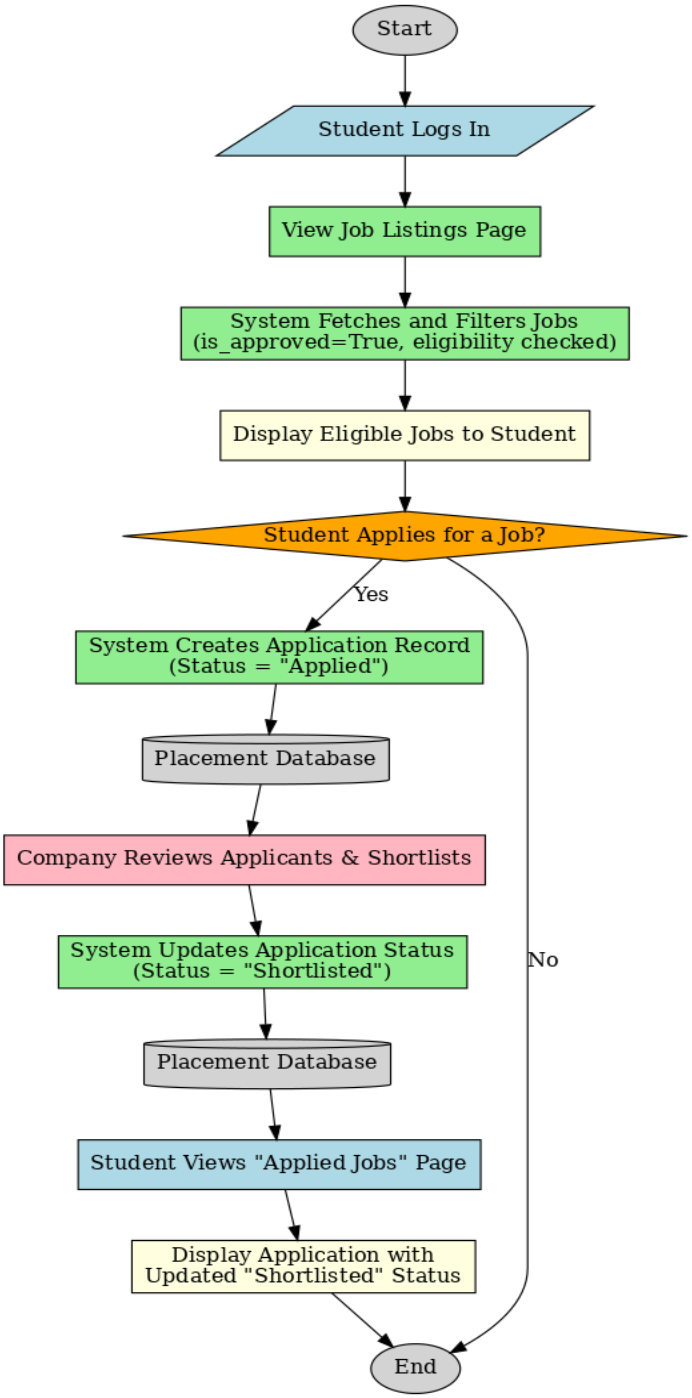


10. FLOWCHART (FLOW OF PROJECT)

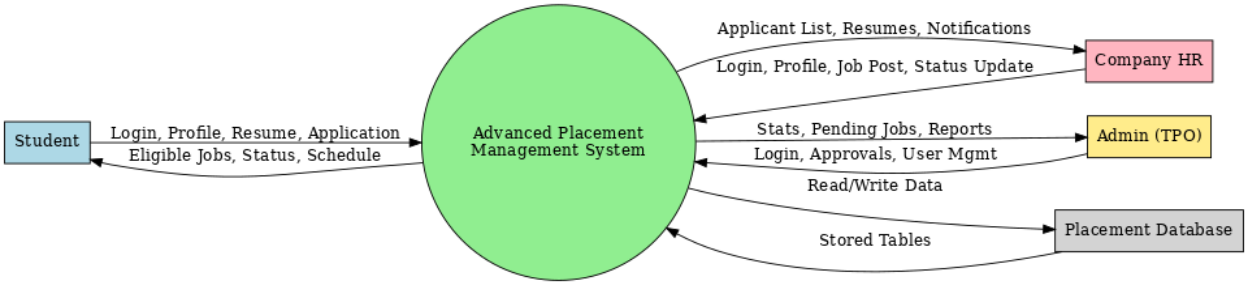
Job Posting & Approval Workflow



Student Application Workflow



High-Level System Data Flow (DFD Level 0)



11. CODE SNIPPETS

- **core/models.py:** The definition of your User, StudentProfile, and JobPosting models.

```
# core/models.py

from django.db import models
from django.contrib.auth.models import AbstractUser

    """
    Custom User model to include a 'role' for differentiating between
    Admins, Students, and Companies.
    """
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('student', 'Student'),
        ('company', 'Company'),
    )
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='student')

# ================================================================================
# 2. Profile Models (Extending the User Model)
# ================================================================================
class StudentProfile(models.Model):
    """
    Stores all data specific to a student. Linked one-to-one with the User model.
    """
    user = models.OneToOneField(User, on_delete=models.CASCADE, primary_key=True, related_name='student_profile')
    phone_number = models.CharField(max_length=15, blank=True, null=True)
    resume = models.FileField(upload_to='resumes/', null=True, blank=True, help_text="Upload your resume in PDF format.")
    cgpa = models.FloatField(null=True, blank=True)
    branch = models.CharField(max_length=100)
    graduation_year = models.IntegerField(null=True)
    backlogs = models.IntegerField(default=0)
    skills = models.TextField(blank=True, help_text="Comma-separated skills (e.g., Python, Django, JavaScript)")
    linkedin_url = models.URLField(blank=True)
    github_url = models.URLField(blank=True)
    is_placed = models.BooleanField(default=False)

    def __str__(self):
        return self.user.username

class CompanyProfile(models.Model):
    """
    Stores all data specific to a company. Linked one-to-one with the User model.
    """
```

```
    user = models.OneToOneField(User, on_delete=models.CASCADE, primary_key=True, related_name='company_profile')
    name = models.CharField(max_length=200)
    description = models.TextField(help_text="A brief description of the company.")
    website = models.URLField(blank=True)
    logo = models.FileField(upload_to='company_logos/', null=True, blank=True)
    is_approved = models.BooleanField(default=False, help_text="Admin must approve the company profile.")
    hr_name = models.CharField(max_length=150, blank=True)
    hr_email = models.EmailField(blank=True)

    def __str__(self):
        return self.name

# ================================================================================
# 3. Core Functional Models
# ================================================================================
class JobPosting(models.Model):
    """
    Represents a job opening posted by a company.
    """
    company = models.ForeignKey(CompanyProfile, on_delete=models.CASCADE, related_name='jobs')
    title = models.CharField(max_length=200)
    description = models.TextField()
    salary_range = models.CharField(max_length=100, blank=True)
    location = models.CharField(max_length=100)
    application_deadline = models.DateField()
    min_cgpa = models.FloatField(default=6.0, help_text="Minimum CGPA required.")
    max_backlogs = models.IntegerField(default=0, help_text="Maximum number of backlogs allowed.")
    allowed_branches = models.CharField(max_length=255, help_text="Comma-separated list of allowed branches (e.g.,
CSE,ECE,ME).")
    is_approved = models.BooleanField(default=False, help_text="Admin must approve the job posting.")
    posted_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.title} at {self.company.name}"

class Application(models.Model):
```

- **core/views.py:** The code for a complex view, like the admin_dashboard view (showing database queries for stats)

```
@login_required
@admin_required
def admin_dashboard(request):
    # This view is already implemented correctly
    student_count = StudentProfile.objects.count()
    placed_count = StudentProfile.objects.filter(is_placed=True).count()
    unplaced_count = student_count - placed_count
    company_count = CompanyProfile.objects.count()
    pending_jobs_count = JobPosting.objects.filter(is_approved=False).count()
    context = {
        'student_count': student_count,
        'placed_count': placed_count,
        'unplaced_count': unplaced_count,
        'company_count': company_count,
        'pending_jobs_count': pending_jobs_count,
    }
    return render(request, 'admin/admin_dashboard.html', context)

@login_required
@admin_required
def manage_students_view(request):
    # This view is already implemented correctly
    students_list = StudentProfile.objects.all().select_related('user').order_by('user__first_name')
    # ... (filtering and pagination logic) ...
    paginator = Paginator(students_list, 15)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    context = {'students_page_obj': page_obj}
    return render(request, 'admin/manage_students.html', context)
```

.

- **core/forms.py:** The code for one of ModelForm classes, like StudentProfileForm.

```
class StudentProfileForm(forms.ModelForm):
    """Form for updating the student-specific profile data."""
    class Meta:
        model = StudentProfile
        exclude = ['user', 'is_placed']
        widgets = {
            'branch': forms.TextInput(attrs={'placeholder': 'e.g., Computer Science Engineering'}),
            'cgpa': forms.NumberInput(attrs={'placeholder': 'e.g., 8.75'}),
            'backlogs': forms.NumberInput(attrs={'placeholder': 'e.g., 0'}),
            'skills': forms.Textarea(attrs={'rows': 3, 'placeholder': 'e.g., Python, Django, React, SQL'}),
            'linkedin_url': forms.URLInput(attrs={'placeholder': 'https://www.linkedin.com/in/yourprofile'}),
            'github_url': forms.URLInput(attrs={'placeholder': 'https://github.com/yourusername'}),
            'phone_number': forms.TextInput(attrs={'placeholder': 'e.g., +919876543210'}),
        }
```

- **core/urls.py:** A section of urlpatterns showing how structure the URLs for one of the panels.

```
# ================================================================================
# 4. Admin Panel URLs
# ================================================================================
path('tpo/dashboard/', views.admin_dashboard, name='admin_dashboard'),
path('tpo/analytics/', views.analytics_view, name='analytics'),
path('tpo/students/', views.manage_students_view, name='manage_students'),
path('tpo/companies/', views.manage_companies_view, name='manage_companies'),
path('tpo/jobs/approve/', views.approve_jobs_view, name='approve_jobs'),
path('tpo/jobs/approve/<int:job_id>/', views.approve_single_job_view, name='approve_single_job'),
path('tpo/jobs/reject/<int:job_id>/', views.reject_single_job_view, name='reject_single_job'),
path('tpo/jobs/all/', views.manage_jobs_view, name='manage_jobs'),
path('tpo/interviews/', views.interview_management_view, name='interview_management'),
path('tpo/reports/', views.generate_reports_view, name='generate_reports'),
path('tpo/reports/export-students-csv/', views.export_students_csv_view, name='export_students_csv'),
path('tpo/reports/export-jobs-csv/', views.export_jobs_csv_view, name='export_jobs_csv'),
path('tpo/upload/', views.bulk_upload_view, name='bulk_upload'),
path('tpo/logs/', views.audit_logs_view, name='audit_logs'),
path('tpo/jobs/approve/', views.approve_jobs_view, name='approve_jobs'),
path('tpo/jobs/approve/<int:job_id>/', views.approve_single_job_view, name='approve_single_job'),
path('tpo/jobs/reject/<int:job_id>/', views.reject_single_job_view, name='reject_single_job'),
```

- **templates/student/job_listings.html:** A snippet showing how loop through jobs and use {% if %} tags to display different buttons.

```
{% extends 'student/student_base.html' %}

{% block student_content %}
<h2 class="fw-bold mb-4"><i class="fas fa-briefcase me-2"></i>Available Job Openings</h2>
<div class="row">
    <div class="col-lg-3"><div class="card shadow-sm mb-4 sticky-top" style="top: 20px;"><div class="card-header"><h5 class="mb-0"><i class="fas fa-filter me-2"></i>Filter Jobs</h5></div><div class="card-body"><form method="get"><div class="mb-3"><label for="companyName" class="form-label fw-bold">Company Name</label><input type="text" class="form-control" name="company_name" id="companyName" value="{{ request.GET.company_name|default:'' }}"></div><div class="mb-3"><label for="jobTitle" class="form-label fw-bold">Job Title</label><input type="text" class="form-control" name="title" id="jobTitle" value="{{ request.GET.title|default:'' }}"></div><div class="d-grid"><button type="submit" class="btn btn-primary"><i class="fas fa-search me-2"></i>Apply Filters</button></div></form></div></div></div>
    <div class="col-lg-9">
        {% for job in eligible_jobs %}
        <div class="card shadow-sm mb-3">
            <div class="card-body"><div class="row"><div class="col-md-8"><h4 class="card-title fw-bold text-primary">{{ job.title }}</h4><h6 class="card-subtitle mb-2 text-muted">{{ job.company.name }}</h6><p class="card-text mb-1"><span class="me-3"><i class="fas fa-map-marker-alt me-2 text-secondary"></i>{{ job.location }}</span><span><i class="fas fa-money-bill-wave me-2 text-secondary"></i>{{ job.salary_range }}</span></p><p class="card-text"><small class="text-danger fw-bold">Apply Before: {{ job.application_deadline|date:"F d, Y" }}</small></p></div><div class="col-md-4 text-md-end align-self-center"><a href="{% url 'core:job_detail' job.id %}" class="btn btn-outline-primary mb-2 w-100">View Details</a>{% if job.has_applied %}<button class="btn btn-warning w-100" disabled><i class="fas fa-check-circle me-2"></i>Applied</button>{% else %}<form method="post" action="{% url 'core:apply_for_job' job.id %}" class="d-inline w-100">{% csrf_token %}<button type="submit" class="btn btn-success w-100">Apply Now</button></form>{% endif %}</div></div></div>
        </div>
        {% empty %}
        <div class="card"><div class="card-body text-center p-5"><i class="fas fa-search-minus fa-3x text-muted mb-3"></i><p class="lead">No eligible job openings found that match your criteria.</p><p class="text-muted">Try adjusting your filters or check back later!</p></div></div>
        {% endfor %}

        {% if ineligible_jobs %}
            <h4 class="mt-5 text-muted">Jobs You Are Not Eligible For</h4>
            {% for job in ineligible_jobs %}
             <div class="card shadow-sm mb-3 bg-light"><div class="card-body opacity-50"><div class="row"><div class="col-md-8"><h5 class="card-title fw-bold text-secondary">{{ job.title }}</h5><h6 class="card-subtitle mb-2 text-muted">{{ job.company.name }}</h6></div><div class="col-md-4 text-md-end align-self-center"><a href="{% url 'core:job_detail' job.id %}" class="btn btn-sm btn-outline-secondary">View Details</a></div></div></div></div>
            {% endfor %}
        {% endif %}
    </div>
</div>
{% endblock %}
```

12. CONCLUSION AND FUTURE WORK

Conclusion

The "PLACEMENT PRO" project has successfully achieved its primary objective of designing and developing a robust, centralized, and automated web application to streamline the campus recruitment process. By leveraging the power and flexibility of the Python-based Django framework, this system effectively addresses the inefficiencies, communication gaps, and administrative burdens inherent in traditional, manual placement workflows. The project serves as a comprehensive digital bridge connecting the three critical stakeholders in the placement ecosystem: the institutional administrators (TPO), the students, and the corporate recruiters.

The core success of this project lies in the implementation of its multi-faceted, role-based architecture. We have successfully engineered three distinct and secure user panels, each tailored with specific functionalities. The **Student Panel** empowers students to take control of their career journey by allowing them to manage a detailed professional profile, upload resumes, and seamlessly browse and apply for jobs that match their specific eligibility criteria. The **Company Panel** provides corporate partners with an efficient toolset to post job openings, manage their public profile, and effectively screen and shortlist candidates from a filtered applicant pool. Overseeing these activities is the **Admin Panel**, a powerful command centre that grants the Training and Placement Officer complete control over user management, job approval moderation, and access to system-wide analytics.

Technologically, the project demonstrates a successful application of the Model-View-Template (MVT) architectural pattern, which has resulted in a clean, maintainable, and scalable codebase. The Django ORM provided a powerful abstraction layer for database operations, while the built-in authentication system ensured a secure foundation for user management and access control. The responsive frontend, built with HTML, CSS, and Bootstrap, guarantees a consistent and accessible user experience across all devices.

In conclusion, this project is more than just a theoretical exercise; it is a practical and deployable solution to a real-world problem. It significantly reduces administrative overhead, eliminates human error in eligibility checking, enhances transparency for students, and provides companies with a streamlined platform for talent acquisition. The system stands as a testament to how modern web technologies can be effectively applied to transform and optimize critical academic processes.

Future Work

- The current system provides a solid and functional foundation for managing campus placements. However, its modular design allows for numerous exciting enhancements that could further elevate its capabilities and user experience. The following areas represent potential avenues for future development:

1. Enhancements in User Interaction & Communication:

- **Real-time Notifications:** Implement Django Channels to provide real-time, on-site notifications to users. For example, a student would receive an instant pop-up alert when their application status is changed to "Shortlisted," rather than waiting for an email or a page refresh.
- **Integrated Chat Module:** Develop a fully functional real-time chat system, allowing for direct and instant communication between the TPO and students, or between a company HR and shortlisted candidates to clarify interview schedules or other queries.
- **Mobile-First Progressive Web App (PWA):** Enhance the frontend to function as a PWA, allowing users to "install" the application on their mobile home screens for a native-app-like experience, including offline access to certain information and push notifications.

2. Intelligence & Automation Features:

- **AI-Based Resume Parser:** Integrate a machine learning library (like spaCy or NLTK) to automatically parse uploaded resumes. This could extract key skills, educational background, and project experience, pre-filling the student's profile and saving them significant time.
- **Job Recommendation Engine:** Develop a recommendation system that suggests suitable job openings to students based on their profile data, including skills, branch, and academic performance. This would provide a more personalized and proactive job-seeking experience.

3. Administrative & Reporting Enhancements:

- **Advanced Data Visualization:** Expand the admin analytics page with more interactive charts and graphs using libraries like D3.js or Plotly. This could include visualizations for company conversion rates, branch-wise placement trends over multiple years, and average salary packages.
- **PDF Report Generation:** Implement a feature for the admin to generate official, professionally formatted placement summary reports in PDF format directly from the system, which can be used for accreditation purposes or official college records.

- **Company Feedback Module:** Create a system where companies can provide structured feedback on candidates after an interview. This valuable data can be used by the TPO to provide targeted guidance to students for future opportunities.

4. Architectural & Deployment Improvements:

- **Asynchronous Task Queues:** Integrate Celery with Redis or RabbitMQ to handle long-running tasks asynchronously. This would be used for sending bulk email notifications or processing large CSV uploads without blocking the main web server, ensuring the UI remains fast and responsive.
- **Containerization:** Containerize the entire application using Docker and Docker Compose. This would standardize the development and production environments, simplifying deployment and ensuring consistency across different machines.
- By pursuing these future enhancements, the PLACEMENT PRO can evolve into an even more powerful, intelligent, and indispensable tool for any modern educational institution.

13. BIBLIOGRAPHY

The implementation of the PLACMENT PRO was made possible through several key technologies and reference materials.

- The official **Django documentation** provided by the Django Software Foundation was vital for understanding the framework's Model-View-Template (MVT) architecture, ORM, authentication system, and form handling.
- The official **Python documentation** from the Python Software Foundation offered clear explanations of core Python concepts and standard libraries used throughout the project.
- For frontend development, resources such as the **Bootstrap 5 documentation** and the **Mozilla Developer Network (MDN)** were used extensively to implement responsive and user-friendly interfaces with HTML, CSS, and JavaScript.
- Community forums like **Stack Overflow** and educational websites like **GeeksforGeeks** were invaluable for troubleshooting practical issues and learning best practices in Django, Python, and database integration.
- The **Chart.js documentation** was referenced for creating the data visualizations on the admin dashboard.